

1 - Os exercícios devem ser realizados INDIVIDUALMENTE.

2 - As resoluções dos exercícios devem estar em ARQUIVOS DIFERENTES. Uma pasta.zip com todos os arquivos deverá ser entregue até às 23:59 do dia 05/06/26 pelo Google Classroom.

3 - Nas descrições das questões, todos os atributos, métodos, classes e/ou interfaces em negrito devem ser explicitamente criados com o MESMO NOME nas soluções. Do contrário, você é livre para escolher qual a assinatura dos mesmos.

4 - Os tipos de entrada e saída explicitados devem ser respeitados.

5 - É permitido criar atributos, métodos, classes e/ou qualquer estrutura auxiliar que você considerar necessária para a resolução do problema. Porém, tudo que for pedido deverá ser implementado, obrigatoriamente.

6 - Os métodos que forem sobrescritos devem ter a anotação @Override.

7 - A correção da lista será feita através da análise individual de cada código, o principal aspecto não se baseia no output correto, mas em uma arquitetura de solução condizente com os princípios da programação orientada a objeto. Logo utilize getters, setters, modificações de visibilidade e todos os demais conceitos estudados em sala de aula na medida que julgarem necessário para a resolução do problema.

8 - Qualquer dúvida ou inconsistência em relação às questões, entrar em contato urgente com os monitores.

9 - Boa sorte e atentem-se ao prazo! 😊

Questão 1 – JavaTunes: Plataforma de Streaming

A startup **JavaTunes** está construindo uma nova plataforma de streaming e precisa unificar diferentes tipos de mídia em um único catálogo. Música, podcast e audiolivro compartilham informações básicas (título, autor e duração), mas cada um é reproduzido de forma diferente. Além disso, alguns conteúdos podem receber avaliações dos usuários, enquanto outros não.

Você foi contratado para modelar a estrutura central do catálogo utilizando **classe abstrata**, **interface**, **sobrescrita** e **sobrecarga de métodos**, além de manter um contador global de reproduções da plataforma.

Requisitos

Classe abstrata: Midia

Representa qualquer item reproduzível do catálogo.

Atributos

- String **titulo**
- String **autor**
- int **duracaoEmSegundos**
- private static int **totalReproducoes** (compartilhado entre todas as mídias da plataforma)

Construtor

- Recebe titulo, autor e duracaoEmSegundos.

Métodos

- public abstract void **reproduzir()**
- public abstract String **tipoDeMidia()**
- public String **getDuracaoFormatada()**, retorna a duração no formato "MM:SS".
- public static int **getTotalReproducoes()**
- Método auxiliar (de visibilidade protegida) que as subclasses devem chamar dentro de seus **reproduzir()** para incrementar o contador estático.

Interface: Avaliavel

Define o comportamento de mídias que aceitam nota dos usuários (1 a 5).

Métodos

- public void **avaliar**(int nota)

- public double **getMediaAvaliacoes()**

Subclasses (todas devem usar @Override)

Musica (estende **Midia**, implementa **Avaliavel**)

- Atributo extra: String **genero**
- Sobrescreve **reproduzir()** e **tipoDeMidia()**.
- Implementa **avaliar** e **getMediaAvaliacoes** armazenando as notas internamente.
- **Sobrecarga (overload):** public void **reproduzir(double velocidade)**
 - Reproduz a música em uma velocidade customizada (ex.: 1.5x, 2.0x).
 - Também deve contar no **totalReproducoes**.

Podcast (estende **Midia**)

- Atributo extra: int **episodio**
- Sobrescreve **reproduzir()** e **tipoDeMidia()**.
- **Não** é avaliável.

AudioLivro (estende **Midia**, implementa **Avaliavel**)

- Atributo extra: String **narrador**
- Sobrescreve **reproduzir()** e **tipoDeMidia()**.
- Implementa **avaliar** e **getMediaAvaliacoes**.

Classe Main

Implemente os seguintes testes:

- Criar pelo menos uma instância de **Musica**, **Podcast** e **AudioLivro**.
- Reproduzir cada uma chamando **reproduzir()**.
- Reproduzir uma **Musica** usando a sobrecarga **reproduzir(double velocidade)**.
- Avaliar a **Musica** e o **AudioLivro** com pelo menos três notas cada e imprimir a média de avaliações de ambos.
- Ao final, imprimir o valor de **totalReproducoes** através do método estático **getTotalReproducoes()**.

Questão 2 - Java Souls

Você ficou responsável por implementar o sistema de descanso em um jogo, onde há diversas opções de locais de descanso para o Personagem, sendo eles **Fogueira** para vida, **FonteMagica** para mana e **Acampamento** para estamina. O sistema é composto pelos seguintes elementos:

Personagem

Classe que representa o personagem que efetuará o descanso.

- Atributos:
 - double vidaAtual**: Representa a quantidade de vida que o personagem está.
 - double vidaMaxima**: Representa a capacidade máxima de vida do personagem.
 - double energiaAtual**: Representa a quantidade de energia que o personagem está.
 - double energiaMaxima**: Representa a capacidade máxima de energia do personagem.
 - TipoEnergia tipo**: Representa o tipo de energia utilizado pelo personagem.
- Métodos:
 - gets e sets para **vidaAtual** e **energiaAtual**.
 - get para **vidaMaxima** e **energiaMaxima**.

TipoEnergia

Enumerador que define o tipo de energia do personagem.

- **MANA**: Utiliza e recupera apenas mana.
- **ESTAMINA**: Utiliza e recupera apenas estamina.

Restauracao

Interface que define os efeitos e durações de um descanso.

- Métodos:
- **public void restaurar(Personagem personagem, double quantidadeRecuperada)**: recupera o personagem com o tipo correspondente de restauração.
- **public void calcularDuracao(double quantidadeRecuperada)**: exibe a quantidade de vida efetiva que foi recuperada assim como a duração desse descanso.

Fogueira

Classe que representa um local de restauração de vida.

- Atributos:
double eficiencia: Representa a taxa de recuperação de vida por segundo proporcionada pela fogueira.
- Métodos:
implementa a interface de **Restauracao**.

FonteMágica

Classe que representa um local de restauração de mana.

- Atributos:
double eficiencia: Representa a taxa de recuperação de mana por segundo proporcionada pela fonte mágica.
- Métodos:
implementa a interface de **Restauracao**.

Acampamento

Classe que representa um local de restauração de estamina.

- Atributos:
double eficiencia: Representa a taxa de recuperação de estamina por segundo proporcionada pelo acampamento.
- Métodos:
implementa a interface de **Restauracao**.

EnergiaIncompatívelException

Deve estender a classe Exception. Chamada quando o personagem tentar restaurar sua energia usando um tipo incompatível.

Main

Implementar testes:

- Restauração de vida
- Restauração de energia bem sucedido
- Restauração de energia incompatível

Questão 3 – Folha de Pagamento Universitária

Uma universidade está modernizando seu sistema de folha de pagamento. A instituição possui diferentes categorias de funcionários, com regras distintas de cálculo de salário e regras distintas de bonificação. Todos compartilham dados básicos (nome, CPF e salário base), mas alguns recebem bônus adicional e outros não.

Você deve modelar essa estrutura utilizando **classe abstrata**, **interface**, **sobrescrita e sobrecarga de métodos**. O sistema também deve manter um **atributo estático** com o total de funcionários cadastrados na universidade.

Requisitos

Classe abstrata: Funcionario

Representa qualquer funcionário da universidade.

Atributos

- String **nome**
- String **cpf**
- double **salarioBase**
- private static int **totalFuncionarios**

Construtor

- Recebe nome, cpf e salarioBase.
- Incrementa **totalFuncionarios** a cada nova instância criada.

Métodos

- public abstract double **calcularSalarioLiquido()**
- public void **augmentarSalario**(double **valor**)
 - Aumenta o **salarioBase** em um valor fixo em reais.
- public void **augmentarSalario**(double **valor**, boolean **ehPercentual**) ← **sobrecarga (overload)**
 - Se **ehPercentual** for **true**, aumenta o **salarioBase** em uma porcentagem (ex.: 10.0 representa 10%).
 - Se for **false**, deve se comportar como a versão de um único parâmetro.
- public static int **getTotalFuncionarios()**
- Getters e setters considerados necessários.

Interface: Bonificavel

Define funcionários que recebem bônus além do salário.

Métodos

- public double **calcularBonus()**

Subclasses (todas devem usar **@Override** em **calcularSalarioLiquido()**)

Professor (estende **Funcionario**, implementa **Bonificavel**)

- Atributo extra: int **titulacao** (1 = Especialista, 2 = Mestre, 3 = Doutor)
- **calcularSalarioLiquido**: salário base com adicional de 10% por nível de titulação, descontando 11% de INSS sobre o total.
- **calcularBonus**: 5% do salário base por nível de titulação.

TecnicoAdministrativo (estende **Funcionario**)

- Atributo extra: int **nivel** (entre 1 e 5)
- **calcularSalarioLiquido**: salário base + (**nivel** × R\$ 200,00), descontando 11% de INSS sobre o total.
- **Não** é bonificável.

Reitor (estende **Funcionario**, implementa **Bonificavel**)

- Atributo extra: double **gratificacaoCargo**
- **calcularSalarioLiquido**: salário base + **gratificacaoCargo**, descontando 11% de INSS sobre o total.
- **calcularBonus**: 20% do salário base.

Classe Main

Implemente os seguintes testes:

- Criar pelo menos um **Professor**, um **TecnicoAdministrativo** e um **Reitor**.
- Armazenar todos numa estrutura do tipo **Funcionario** (polimorfismo) e imprimir o salário líquido de cada um.
- Em um funcionário, chamar **augmentarSalario(double)** com valor fixo; em outro, chamar **augmentarSalario(double, boolean)** com **ehPercentual = true**. Reimprimir o salário líquido após cada aumento.
- Imprimir o bônus apenas dos funcionários que implementam **Bonificavel** (usar **instanceof**).
- Imprimir o total de funcionários cadastrados utilizando o método estático **getTotalFuncionarios()**.

Questão 4 – Javatouille

Você é responsável por desenvolver um sistema de simulação que representa a cozinha de determinado restaurante. O sistema deve coordenar o trabalho entre Aprendizes e Chefes usando um Balcão compartilhado, onde os aprendizes preparam os ingredientes a serem utilizados pelos Chefes em suas receitas. O sistema é implementado seguindo o paradigma orientado a objetos e concorrência, sendo composto por 4 classes principais: Ingrediente, Balcao, Aprendiz e Chefe.

Ingrediente: Representa os ingredientes manipulados.

- Atributos:
string nome: Representa os ingredientes manipulados.

Balcao: Classe que representa o local compartilhado para a disposição dos ingredientes (o buffer).

- Atributos:
string nome: Representa o nome do balcão.
int capacidade: Representa a quantidade de espaço para ingredientes na bancada. É necessário também implementar uma fila para os ingredientes que estão no balcão
- Métodos:
public void colocar(Ingrediente ingrediente) throws InterruptedException: método utilizado pelos aprendizes para colocar os ingredientes prontos para uso no balcão.

public void retirar(Ingrediente ingrediente) throws InterruptedException: método utilizado pelos chefes para colocar os ingredientes prontos para uso no balcão.

Aprendiz:

- Atributos:
Balcao balcao: Referência ao **Balcao** utilizado na cozinha
int desempenho: Representa a quantidade de ingredientes preparados pelo aprendiz por iteração.
- Métodos:
Implementa a interface Runnable e simula a produção de ingredientes.

Chefe:

- Atributos:
Balcao balcao: Referência ao **Balcao** utilizado na cozinha
int desempenho: Representa a quantidade de ingredientes prontos consumidos pelo Chefe por iteração.
- Métodos:
Implementa a interface Runnable e simula o consumo dos ingredientes.

Main:

Implementar casos

- Caso de balcão com pouco espaço
- Caso de capacidade de produção dos aprendizes muito superior a de chefes
- Caso de capacidade de consumo dos chefes muito superior à dos aprendizes